

C Programming — Day 1 Notes

These notes are written for a **complete beginner**.

You do not need to know any programming before starting.

1. Software

What is Software?

Software is a set of instructions that tells a computer what to do.

In simple words:

Software = Programs that make a computer perform tasks.

Examples

- Google Chrome → used for browsing the internet
- VLC → used for playing videos
- Microsoft Word → used for writing documents
- Calculator → used for calculations

Basic Types of Software

1. System Software

Helps the computer itself work.

Examples:

- Windows
- Linux
- macOS

2. Application Software

Helps the user perform a specific task.

Examples:

- Chrome
 - MS Word
 - VLC
 - Calculator
-

2. Programming Language

What is a Programming Language?

A **programming language** is a language used to give instructions to a computer.

Humans use languages like:

- English
- Hindi
- Gujarati

Computers can be programmed using languages such as:

- C
- C++
- Java
- Python
- JavaScript

What is C?

C is a programming language.

It is used to create:

- Software
- Operating systems
- Embedded systems
- Games
- System programs

Example:

```
printf("Hello");
```

This tells the computer to display:

```
Hello
```

3. Algorithm

What is an Algorithm?

An **algorithm** is a **step-by-step procedure to solve a problem**.

Think about making tea.

You don't randomly make tea. You follow steps.

Real-Life Example: Making Tea

Algorithm:

1. Start
2. Take water
3. Boil the water
4. Add tea powder
5. Add milk
6. Add sugar
7. Boil the tea
8. Serve the tea
9. End

The important idea is:

Algorithm = Step-by-step solution

Programming Example

Problem:

Find the sum of two numbers.

Algorithm:

1. Start
2. Take first number
3. Take second number
4. Add the two numbers
5. Display the result
6. End

4. Flowchart

What is a Flowchart?

A **flowchart** is a diagram that shows the steps of an algorithm.

Instead of writing only text, we use **shapes and arrows**.

Why do we use a Flowchart?

It helps us:

- Understand the problem
- Understand the order of steps

- Find mistakes easily
- Plan a program before coding

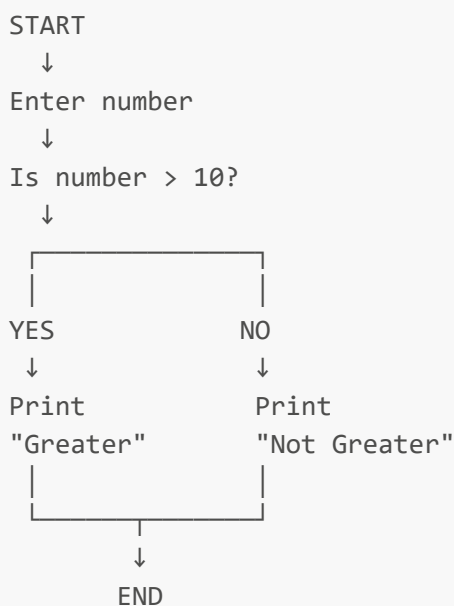
Common Flowchart Symbols

Symbol	Name	Purpose
Oval	Start/End	Shows beginning or ending
Rectangle	Process	Shows an action
Parallelogram	Input/Output	Taking or displaying data
Diamond	Decision	Checks a condition

Simple Example

Problem:

Check whether a number is greater than 10.



5. Structure of a C Program

Before writing C programs, we should understand the basic structure.

A simple C program:

```
#include <stdio.h>

int main()
{
```

```
int age;

printf("Enter your age: ");
scanf("%d", &age);

return 0;
}
```

Don't worry if this looks new.

We will understand each part one by one.

5.1 Header File

```
#include <stdio.h>
```

What is a Header File?

A **header file** contains useful information and functions that we can use in our program.

`stdio.h` means:

Standard Input Output

It provides functions such as:

- `printf()`
- `scanf()`

Why do we use it?

Because we need `printf()` and `scanf()` to display and take input.

5.2 `main()`

```
int main()
{
}
```

What is `main()`?

`main()` is the **starting point of a C program**.

When we run a C program, execution starts from `main()`.

Think of it as:

main() = Starting point of the program

5.3 Variable Declaration

Example:

```
int age;
```

This creates a variable called `age`.

We will learn variables in detail later.

For now, remember:

Declaration = Telling C about a variable before using it.

5.4 Statements

A **statement** is an instruction given to the computer.

Example:

```
printf("Hello");
```

This is a statement.

Another example:

```
age = 20;
```

Most C statements end with:

```
;
```

This semicolon is called a **statement terminator**.

5.5 `return 0`

```
return 0;
```

This tells the operating system that the program finished successfully.

For now, simply remember:

```
return 0; → Program ended successfully.
```

6. `printf()`

What is `printf()`?

`printf()` is a C function used to **display output on the screen**.

Why do we use it?

Whenever we want to show something to the user.

Basic Syntax

```
printf("message");
```

Example

```
printf("Hello");
```

Output:

```
Hello
```

Printing a Number

```
printf("%d", 10);
```

Output:

```
10
```

Printing Multiple Things

```
printf("Age = %d", 20);
```

Output:

```
Age = 20
```

7. scanf()

What is scanf()?

scanf() is a C function used to **take input from the user**.

For example, the program can ask:

```
Enter your age:
```

The user enters:

```
20
```

The program can store 20 in a variable.

Basic Syntax

```
scanf("format specifier", &variable);
```

Example:

```
int age;  
  
scanf("%d", &age);
```

Here:

- %d → tells C that we are taking an integer
- &age → tells C where to store the entered value

For now, remember:

```
printf() → gives output  
scanf() → takes input
```

Simple Example

```
int age;  
  
printf("Enter your age: ");  
scanf("%d", &age);  
  
printf("Your age is %d", age);
```

If the user enters:

20

Output:

```
Enter your age: 20  
Your age is 20
```

8. Format Specifiers

What is a Format Specifier?

A **format specifier** tells C what type of data we are working with.

It starts with %.

Important Format Specifiers

Specifier	Used For	Example
%d	Integer	10
%f	Float	10.5
%c	Character	'A'
%s	String	"Hello"
%lf	Double	10.5678

Examples

Integer

```
int age = 20;

printf("%d", age);
```

Float

```
float price = 25.5;

printf("%f", price);
```

Character

```
char grade = 'A';

printf("%c", grade);
```

String

```
char name[] = "Rahul";

printf("%s", name);
```

Double

```
double value = 25.6789;

printf("%lf", value);
```

9. Escape Sequences

What is an Escape Sequence?

An **escape sequence** is a special combination of characters that performs a special action.

Escape sequences usually start with:

\

Important Escape Sequences

Escape Sequence	Meaning
<code>\n</code>	New line
<code>\t</code>	Tab space
<code>\"</code>	Print double quote
<code>\\</code>	Print backslash

`\n` — New Line

```
printf("Hello\nWorld");
```

Output:

```
Hello
World
```

`\t` — Tab

```
printf("Hello\tWorld");
```

Output:

```
Hello    World
```

`\"` — Double Quote

```
printf("He said \"Hello\"");
```

Output:

```
He said "Hello"
```

\\ — Backslash

```
printf("C:\\Program");
```

Output:

```
C:\Program
```

10. Data Types

What is a Data Type?

A **data type** tells C what kind of data a variable will store.

For example:

- Whole number → `int`
- Decimal number → `float`
- Character → `char`

Common Data Types

Data Type	Used For	Example
<code>int</code>	Whole numbers	<code>10</code>
<code>float</code>	Decimal numbers	<code>10.5</code>
<code>double</code>	More precise decimal numbers	<code>10.56789</code>
<code>char</code>	Single character	<code>'A'</code>

10.1 `int`

Used for whole numbers.

```
int age = 20;
```

Examples:

```
10  
25  
100  
-5
```

10.2 float

Used for decimal numbers.

```
float price = 99.50;
```

Examples:

```
10.5  
25.75  
99.99
```

10.3 double

Also used for decimal numbers.

It can store decimal values with greater precision than `float`.

```
double pi = 3.1415926535;
```

10.4 char

Used for **one character**.

```
char grade = 'A';
```

Examples:

```
'A'  
'B'  
'5'  
'#'
```

A character is written inside **single quotes**.

```
'A'
```

Not:

```
"A"
```

11. Variables

What is a Variable?

A **variable** is a named memory location used to store a value.

For a beginner, think of it as:

Variable = A named box that stores data.

Example:

```
int age;
```

Here, `age` is a variable.

We can store a value inside it:

```
age = 20;
```

Now the variable contains:

```
age → 20
```

11.1 Declaration

What is Declaration?

Declaration means telling C:

"I want to create a variable of this type."

Example:

```
int age;
```

Here:

- `int` → data type
- `age` → variable name

Another example:

```
float price;
```

11.2 Initialization

What is Initialization?

Initialization means giving a variable its first value when creating it.

Example:

```
int age = 20;
```

Here we create `age` and immediately give it `20`.

Another example:

```
float price = 99.50;
```

11.3 Assigning a Value

We can give a value to a variable after declaring it.

```
int age;  
  
age = 20;
```

First:

```
int age;
```

Then:

```
age = 20;
```

11.4 Changing a Value

A variable's value can be changed.

```
int age = 20;  
  
age = 21;
```

Initially:

```
age = 20
```

After:

```
age = 21
```

The old value is replaced.

Quick Revision

Concept	Simple Meaning
Software	Instructions/programs used by a computer
Programming Language	Language used to give instructions to a computer
C	A programming language
Algorithm	Step-by-step solution
Flowchart	Diagram of an algorithm
<code>main()</code>	Starting point of a C program
Header file	Provides useful declarations/functions
<code>printf()</code>	Displays output
<code>scanf()</code>	Takes input

Concept	Simple Meaning
Format Specifier	Tells C what type of data is being used
Data Type	Tells C what kind of data to store
Variable	Named place used to store data
<code>return 0</code>	Shows successful program completion

Final Practice Program

Here is a **tiny complete C program** using the concepts we learned.

```
#include <stdio.h>

int main()
{
    int age;
    float height;

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height: ");
    scanf("%f", &height);

    printf("\nAge = %d\n", age);
    printf("Height = %.2f\n", height);

    return 0;
}
```

Sample Output

```
Enter your age: 20
Enter your height: 5.8

Age = 20
Height = 5.80
```

Line-by-Line Explanation

Line 1

```
#include <stdio.h>
```

We include the `stdio.h` header file.

It gives us functions such as:

- `printf()`
- `scanf()`

Line 2

```
int main()
```

This is the starting point of our program.

The program starts running from `main()`.

Line 3

```
{
```

This marks the beginning of the `main()` block.

The instructions inside `{ }` belong to `main()`.

Line 4

```
int age;
```

We create a variable called `age`.

Its data type is `int`.

So it can store a whole number such as:

```
18  
20  
25
```

Line 5

```
float height;
```

We create another variable called `height`.

Its data type is `float`.

It can store decimal values such as:

```
5.5  
5.8  
6.2
```

Line 6

```
printf("Enter your age: ");
```

`printf()` displays a message on the screen.

The user sees:

```
Enter your age:
```

Line 7

```
scanf("%d", &age);
```

`scanf()` takes input from the user.

- `%d` → input is an integer
- `&age` → store the entered number in `age`

If the user enters:

```
20
```

then:

```
age = 20
```

Line 8

```
printf("Enter your height: ");
```

This asks the user to enter their height.

Line 9

```
scanf("%f", &height);
```

This takes a decimal value from the user.

- `%f` → float
- `&height` → store the value in `height`

If the user enters:

```
5.8
```

then:

```
height = 5.8
```

Line 10

```
printf("\nAge = %d\n", age);
```

This displays the age.

`%d` is replaced by the value stored in `age`.

The `\n` moves the cursor to a new line.

If:

```
age = 20
```

the output becomes:

```
Age = 20
```

Line 11

```
printf("Height = %.2f\n", height);
```

This displays the height.

`%f` is used for a float.

`.2` means:

Display **2 digits after the decimal point**.

So:

```
5.8
```

is displayed as:

```
5.80
```

The `\n` moves to the next line.

Line 12

```
return 0;
```

This tells the computer that the program finished successfully.

Line 13

```
}
```

This marks the end of `main()`.

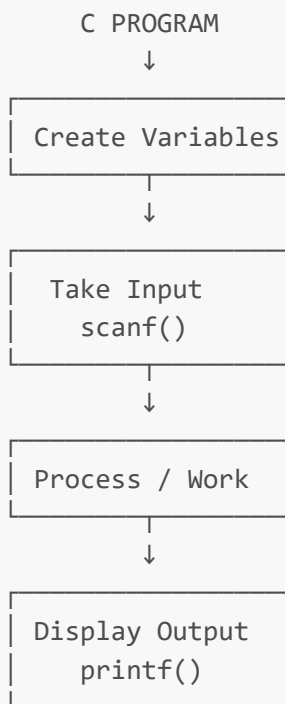


Remember These 5 Things First

For your **first C programming class**, focus on these:

1. `printf()` → Output
2. `scanf()` → Input
3. Variable → Stores data
4. Data Type → Tells what kind of data
5. Format Specifier → Tells C how to handle the data

The basic flow is:



This is the basic foundation you need before moving to operators, conditions, loops, arrays, functions, and other C topics.